UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

GROUP PROJECT REPORT

LaTeX Generation for Mathematical Expressions: An experiment with different models

---

Group 32

Full Name	Student ID	Major
Nguyen Quang Minh [Leader]	22BA13221	ICT
Nguyen Cao Manh Thang	23BI14403	MAT
Le Sy Han	23BI14150	DS
Le Hoang Dat	23BI14087	DS
Bui Tuan Thanh	22BA13284	ICT
Duong Duc Anh	23BI14012	ICT

Supervisor: MSc. Huynh Vinh Nam

January, 2026

Contents

List of Abbreviations **2**

1 Introduction **4**

2 Methodology **5**
2.1 Pre-processing the Data & Preparing the Dictionary for Training . . . 5
2.2 Model Implementation . . . 6
2.2.1 CNN-Long Short-Term Memory . . . 6
2.2.2 Vision Transformer (ViT) . . . 8
2.2.3 Convolutional Transformer . . . 10
2.3 Model Training . . . 11
2.3.1 Environment . . . 11
2.3.2 Hyperparameter Settings . . . 11
2.3.3 Loss Function . . . 11
2.3.4 Training Strategy . . . 12
2.4 Model Testing . . . 12
2.5 Model Evaluation . . . 13
2.5.1 Token-level Levenshtein Distance . . . 13
2.5.2 BLEU Score . . . 15
2.6 Website Implementation . . . 16
2.6.1 Development Framework and Technology Stack . . . 16
2.6.2 Backend Implementation (Server-Side) . . . 17
2.6.3 Frontend Interaction and Asynchronous Handling . . . 17
2.6.4 System Workflow . . . 17

3 Results **18**
3.1 Model Parameter Comparison . . . 18
3.2 Model Training and Validation Results . . . 18
3.3 Test Set Evaluation . . . 20
3.4 Model Deployment Results on Website . . . 21
3.4.1 Website Overview . . . 21
3.4.2 User Interaction . . . 21
3.4.3 Example Result . . . 22

4 Discussion **23**
4.1 On the Success Rate of the Model . . . 23
4.2 Future Perspectives . . . 23

5 Summary **23**

References **25**

Appendix: Source Code Repository **25**

List of Figures

1 Sample entries from the Dataset. Pictures from HuggingFace 4
2 The overall pipeline of the proposed CNN-LSTM architecture. 6
3 Architecture of the CNN encoder for feature extraction. 6
4 Architecture of the CNN-LSTM Encoder-Decoder network. 7
5 The overall pipeline of the proposed ViT-Transformer architecture. 8
6 Overview of the Vision Transformer (ViT) architecture. 9
7 Overview of the Transformer Decoder architecture. 10
8 Overview of the Convolutional Transformer Encoder architecture. 11
9 CNN-LSTM Model: Training and Validation Performance Curves 18
10 ViT Model: Training and Validation Performance Curves 19
11 Convolutional Transformer Model: Training and Validation Performance Curves 19
12 Web application interface displaying prediction results from three models: CNN-LSTM, ViT, and Convolutional Transformer. 21

List of Tables

1 Hyperparameter settings used for model training. 11
2 Number of learnable parameters for each model 18
3 Test Set Evaluation 20
4 Prediction Results Comparison 22

List of Abbreviations

CNN	Convolutional Neural Network
LSTM	Long Short-Term Memory
ViT	Vision Transformer
AI	Artificial Intelligence
ML	Machine Learning
MLP	Multilayer Perceptron

Abstract

In this work, we experiment with different models for the problem of generating LaTeX code for Mathematical Expressions. The models considered in this paper all use the Encoder-Decoder architecture: CNN-LSTM, pure Vision Transformer (ViT), and Convolutional Transformer. After training, we evaluate these three models on two metrics: Normalized Levenshtein Distance and BLEU Score. Experimental results show that the Convolutional Transformer model consistently outperforms both CNN–LSTM and ViT, demonstrating good generalization and ability of generating LaTeX code from images of mathematical expressions. Based on these findings, we discuss the strengths and limitations of each architecture. Finally, the trained models are deployed on a web-based platform, allowing users to interact with and experiment with the systems.

Key Words: Natural Language Processing, LaTeX Code Generator, LaTeX dictionary, Transformer-based Models, Hybrid Models, Text Generation Evaluation, UI/UX Implementation

Acknowledgements

We would like to express our sincere gratitude to our supervisor, PhD. Huynh Vinh Nam, for his valuable guidance, feedback, and encouragement throughout this project. His support has helped us better understand the relevant knowledge and successfully complete this work.

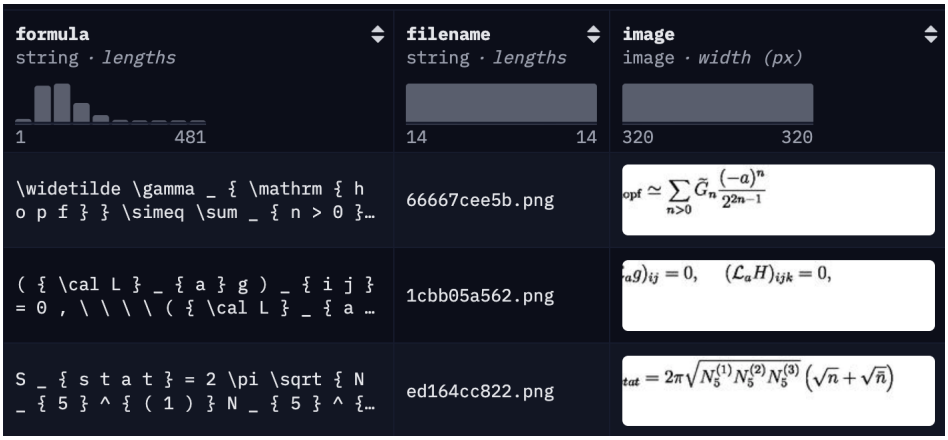
We are also very grateful to Dr. Nguyen Bich Van for inspiring the idea behind this project and for supporting us in the model training progress. Her direction and support have been essential to the development of our work.

1 Introduction

LaTeX generation for mathematical expressions is an active research topic in machine learning and artificial intelligence due to its inherently interdisciplinary nature. The task simultaneously involves computer vision, natural language processing, and sequence generation, as it requires translating visual representations of mathematical expressions into structured LaTeX code. Successfully addressing this problem enables automatic conversion of rendered mathematical expressions into LaTeX format, significantly reducing the effort required for researchers and students to produce LaTeX documents.

Numerous approaches have been proposed in the literature, including those presented in [3] and [5]. Most existing methods adopt an encoder–decoder framework, with two dominant architectural choices: CNN – LSTM based and Vision Transformer–based models. In this group project, we implement three models — corresponding to the two aforementioned architectures and a hybrid Convolutional Transformer, and then systematically analyze their respective strengths and limitations.

The data set used in this study is Im2LaTeX, which consists of approximately 100,000 images of mathematical expressions paired with their corresponding LaTeX code. The data set is partitioned into training, validation, and test subsets. Each data entry comprises two fields: an image filename and the associated LaTeX sequence. Representative examples of the data set are presented below.



formula string · *lengths* 1 481	filename string · *lengths* 14 14	image image · *width (px)* 320 320
\widetilde \gamma _ { \mathrm { h o p f } } \simeq \sum _ { n > 0 }…	66667cee5b.png	$\text{opf} \simeq \sum_{n>0} \widetilde{G}_n \frac{(-a)^n}{2^{2n-1}}$
({ \cal L } _ { a } g) _ { i j } = 0 , \ \ \ \ ({ \cal L } _ { a …	1cbb05a562.png	$_a g)_{ij} = 0, \quad (\mathcal{L}_a H)_{ijk} = 0,$
S _ { s t a t } = 2 \pi \sqrt { N _ { 5 } ^ { (1) } N _ { 5 } ^ {…	ed164cc822.png	$_{tat} = 2\pi\sqrt{N_5^{(1)} N_5^{(2)} N_5^{(3)}} \left(\sqrt{n} + \sqrt{\bar{n}}\right)$

Figure 1: Sample entries from the Dataset. Pictures from HuggingFace

2 Methodology

The project includes multiple stages:

- Pre-processing the Data & Preparing the Dictionary for Training
- Model Implementation
- Model Training
- Model Testing
- Model Evaluation
- Website Implementation

2.1 Pre-processing the Data & Preparing the Dictionary for Training

The task begins by identifying the fundamental units of LaTeX code that a model must generate when provided with an image of a mathematical expression. To address this, we construct a vocabulary of LaTeX tokens by extracting all LaTeX sequences from the dataset, including both the training and validation sets. This process results in a dictionary of LaTeX components.

We then define the categories of tokens that may appear in a LaTeX expression as follows:

- `<sos>` | `<eos>` : start and end of sequence
- `(\\[a-zA-Z]+)` — LaTeX commands such as `\frac`, `\sum`, `\int`
- `([{}_^$%&#])` — common LaTeX symbols such as `{`, `^`, `_`
- `([0-9])` — a single digit (e.g., 1, 2, 3)
- `([a-zA-Z])` — a single letter (e.g., x, y, a)
- `(\S)` — any non-whitespace character (e.g., =, +, -)

Our objective is to identify LaTeX expressions according to the token categories introduced above. We employ regular expressions, which describe classes of character sequences that follow a common pattern. In Python, the "`re`" library provides appropriate tools for pattern matching and extraction within text.

Using regular expressions, we construct the LaTeX token dictionary through the following steps:

- Use the function `findall` in `re`, which will return all non-overlapping matches of pattern as a list. The patterns are defined in a string and passed together with the LaTeX source text using the code: `tokens = re.findall(pattern, text)`
- Import from the `torchtext.vocab` library the function `build_vocab_from_iterator` in a different cell. In `torchtext`, `Vocab` can be understood as a function that maps tokens to indices (which are numbers substituting for the tokens, or in this case, words). This function is necessary as later, we will feed the model a chain of LaTex texts with a vector - which can only have numbers.
- Form the vocabulary from the predefined token set along with four special symbols.

  - The <unk> token is introduced to represent any out-of-vocabulary symbol and thus marks mathematical expressions that do not appear in the LaTeX training dataset.
  - The <pad> token provides a placeholder used to equalize sequence lengths during batching.
  - The <sos> and <eos> tokens define the boundaries of each sequence, indicating the beginning and the end of the code the model is expected to generate.

- Pass the list tokens (including the special ones above) to the function `_vocab_from_iterator`. The default index is set to the <unk> token. We now have a dictionary for the task.

2.2 Model Implementation

2.2.1 CNN-Long Short-Term Memory

A unified CNN architecture is used as a shared backbone to learn features from images and an LSTM decoder is utilized to align these visual features with the predicted mathematical tokens.

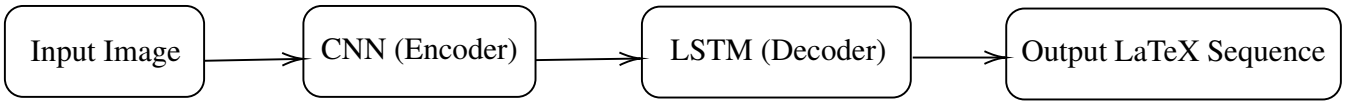


Figure 2: The overall pipeline of the proposed CNN-LSTM architecture.

CNN Encoder Architecture: The basic structure is a series of convolutional blocks that consists of convolutional and pooling layers, followed by a set of fully connected layers. This hierarchical structure allows the model to learn low-level features (such as edges and curves) in early layers and high-level abstract representations of mathematical symbols in deeper layers. After several convolutional layers, the output is a vector with length of 128, representing the encoded visual context of the mathematical expression.

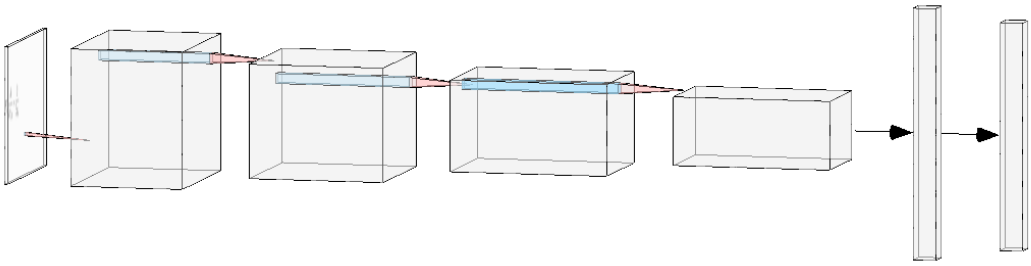


Figure 3: Architecture of the CNN encoder for feature extraction.

LSTM Decoder: After the feature extraction phase, the sequence decoding module translates the compressed visual representation into a corresponding sequence of LaTeX tokens. We implemented a LstmDecoder class inheriting from PyTorch's nn.Module.

- **Token Embedding:** The discrete LaTeX tokens $\mathbf{w_t}$ are mapped to dense vectors $\mathbf{e}_t \in \mathbb{R}^D$ using a learnable embedding matrix. The embedding dimension is set to $D = 128$, matching the dimension of the feature vector.

- **Feature Vector Replication:** To provide visual context, the global image feature vector $\mathbf{v} \in \mathbb{R}^D$ (obtained from the CNN Encoder) is replicated T times to match the sequence length.

- **Concatenation:** Input of LSTM decoder $\mathbf{x_t}$ is concatenation of feature vector and token embedding.

$$\mathbf{x}_t = [\mathbf{e}_t; \mathbf{v}] \in \mathbb{R}^{2D}$$

- **Recurrent Processing:** The core of the decoder is a Long Short-Term Memory (LSTM) network. Given the concatenated input x_t, the LSTM recursively updates its internal states. We initialize the first hidden state (h_0) and cell state (c_0) with zeros. Then, after each calculation, the internal gates of the LSTM update these states based on the current concatenated input and the previous states. The resulting h_t is used both for predicting the current token and as the recurrent input for the next time step.

$$\mathbf{h}_t, \mathbf{c}_t = \text{LSTM}(\mathbf{x}_t, \mathbf{h}_{t-1}, \mathbf{c}_{t-1})$$

- **Token Prediction:** The hidden state h_t at each time step serves as a high-level representation of the predicted character in its context. To map this representation back to the vocabulary space, we apply a fully connected linear transformation:

$$\mathbf{z}_t = \mathbf{W}\mathbf{h}_t + \mathbf{b}$$

The output z_t represents the logits, which are then processed by a Softmax function to obtain the probability distribution $\mathbf{p_t}$ over the vocabulary for the next token prediction.

$$\mathbf{p_t} = \text{Softmax}(\mathbf{z_t})$$

Finally, the predicted token for the current time step is determined by selecting the index with the highest probability from $\mathbf{p_t}$. This predicted token $\mathbf{w_t}$ is then fed into the embedding layer of the next time step.

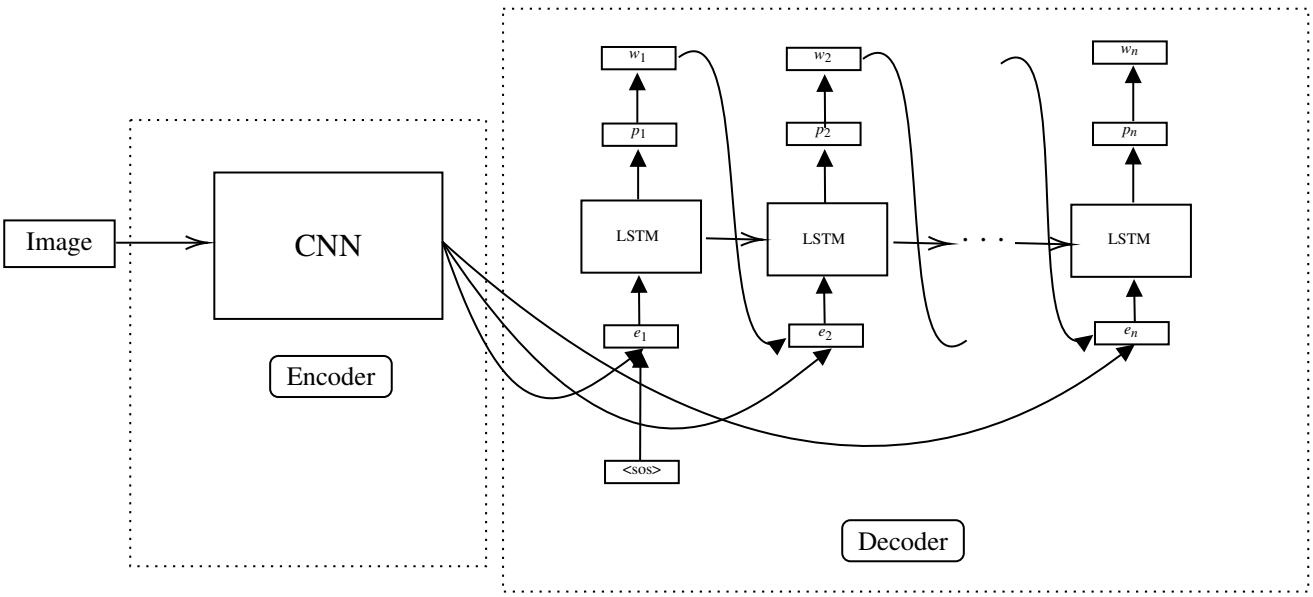


Figure 4: Architecture of the CNN-LSTM Encoder-Decoder network.

2.2.2 Vision Transformer (ViT)

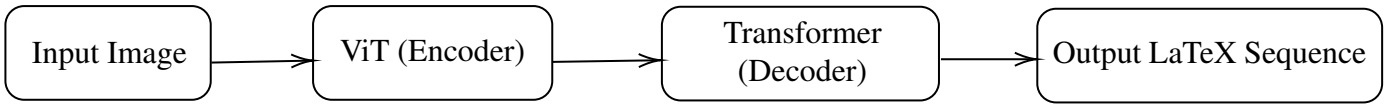


Figure 5: The overall pipeline of the proposed ViT-Transformer architecture.

Vision Transformer Encoder [3]: In contrast to CNN Encoder, which focuses on local features, Vision Transformer (ViT) utilizes the Self-Attention mechanism that enables it to capture global context from the first layer. This characteristic is useful for the recognition of long and complex formulas. In this model, the PreNorm version of Transformer is used (which means the Layer Normalization happens before the attention and feedforward layers rather than after). The implementation is developed in PyTorch and consists of three primary components: Patch Extraction, Embedding Projection, and a stack of Transformer Encoder layers.

- **Patch Extraction:** Instead of creating a feature vector from the image directly, the encoder receives the preprocessed input image x and subdivides it into a sequence of flattened 2D patches. The number of patches $\mathbf{N}$ depend on patch size $\mathbf{P}$ and image size $(\mathbf{H}, \mathbf{W})$, following below formula:

$$\mathbf{N} = \frac{\mathbf{H} \times \mathbf{W}}{\mathbf{P}^2}$$

  This process transforms the 2D image into a sequence of flattened 1D vectors:

$$x_p \in \mathbb{R}^{\mathbf{N} \times \mathbf{P}^2}$$

- **Embedding Projection:** The patch embedding module maps the raw pixel values into feature space. The processing pipeline includes three steps:
  - **Linear Projection:** Each flattened patch is passed through a learnable linear layer to project to feature space with smaller dimension($\mathbf{D} = 128$).
  - **CLS Token:** To aggregate global image information, we append a special learnable classification token ([CLS]) to the beginning of the patch sequence.
  - **Position Embedding:** To retain spatial information, learnable position embeddings $E_{pos} \in \mathbb{R}^{(\mathbf{N}+1) \times \mathbf{D}}$ are added element-wise to the patch embeddings.

  The final input to the transformer is formulated as:

$$z_i = [x_{cls}, x_p^1 E, \ldots, x_p^N E] + E_{pos}$$

- **ViT Layer:** The fundamental building block of ViT is the Transformer encoder layer. The full encoder is constructed by stacking this layer multiple times. The Transformer encoder consists of alternating layers of multiheaded self-attention and MLP blocks. Layernorm (LN) is applied before every block, and residual connections after every block. The Self-Attention mechanism to allow every part of the image to interact with every other part simultaneously. The MLP is located after the self-attention layer, which processes and transforms the features of each patch to extract deeper abstract representations. After several transformer layers, the feature vector evolves into a high-level semantic representation that encapsulates both local visual details and global structural dependencies.

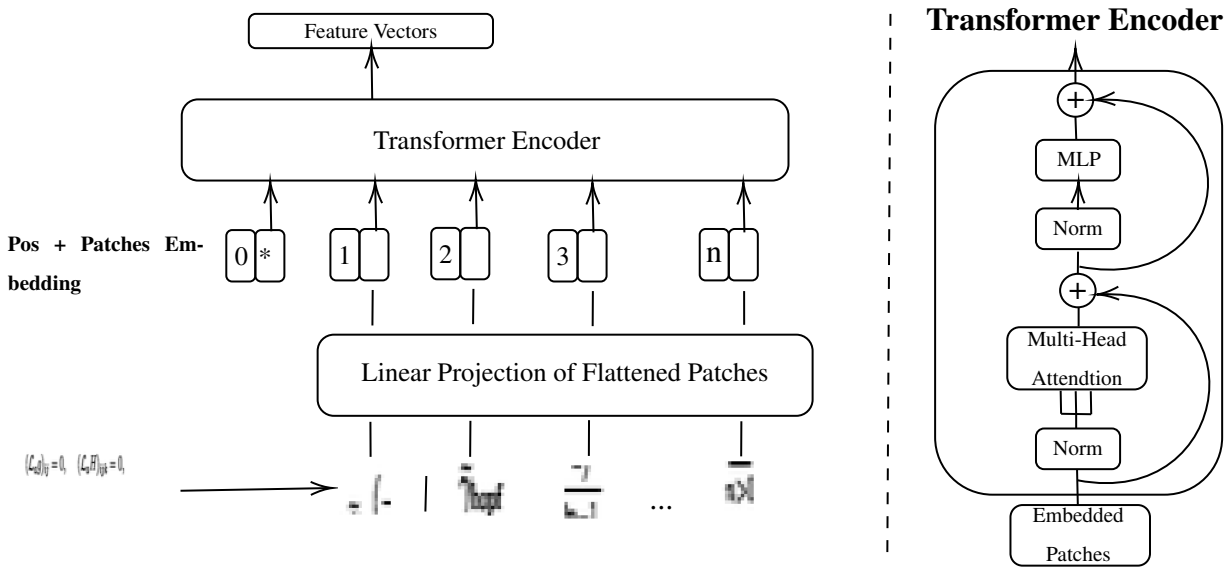


Figure 6: Overview of the Vision Transformer (ViT) architecture.

Transformer Decoder [1]: After capturing features from the images with ViT Encoder, the encoder output will be passed through the Decoder to generate the logits for tokens predicting. The architecture with PreNorm version of Transformer and Cross-Attention mechanism [4] is inspired by the Encoder-Decoder Transformer architecture introduced in Chapter 12 of [1]. The implementation is based on `torch.nn` library in the PyTorch Framework, and contains two main parts: Sequence Embedding and a stack of Transformer Decoder layers.

- **Sequence Embedding:** Unlike RNN/LSTM, which just needs the token embeddings for text sequences, Transformer needs both the token embeddings and positional embeddings. Each discrete LaTeX tokens are mapped to a dense vectors $\mathbf{e}_t \in \mathbb{R}^D$ using a learnable embedding matrix. The positional indices are generated as a sequence from 0 to $T-1$, where T denotes the sequence length, and passed through a positional embedding layer, the result is a learnable positional embedding $\mathbf{p}_t \in \mathbb{R}^D$ to each token embedding. The embedding dimension is set to $D = 128$, matching the dimension of the feature vector. Finally, the token embeddings and their corresponding positional embeddings are added to obtain the sequence embeddings. This combined embedding preserves both semantic and order information of the input sequence.

$$\mathbf{x}_t = \mathbf{e}_t + \mathbf{p}_t.$$

- **Transformer Decoder Layer:** Each decoder layer contains a Causal Self Multi-head Attention sub-layer, a Cross Attention sub-layer and a Feed Forward sub-layer. Layer-Norms are applied before every sub-layer. First, each sequence embedding $\mathbf{x}_t$ of the LaTeX sequences is passed through the Causal Self Multi-head Attention, where the decoder builds contextual representations of LaTeX tokens. After that, the output from the previous sub-layer and the output from the ViT will be combined and then be passed through the Cross Attention, which allows the decoder to attend to each of the source LaTeX tokens as projected into the entire encoder final output representations. Finally, the output will be passed through the Feed Forward sub-layer to generate the logits for prediction. The full decoder is constructed by stacking this layer multiple times.

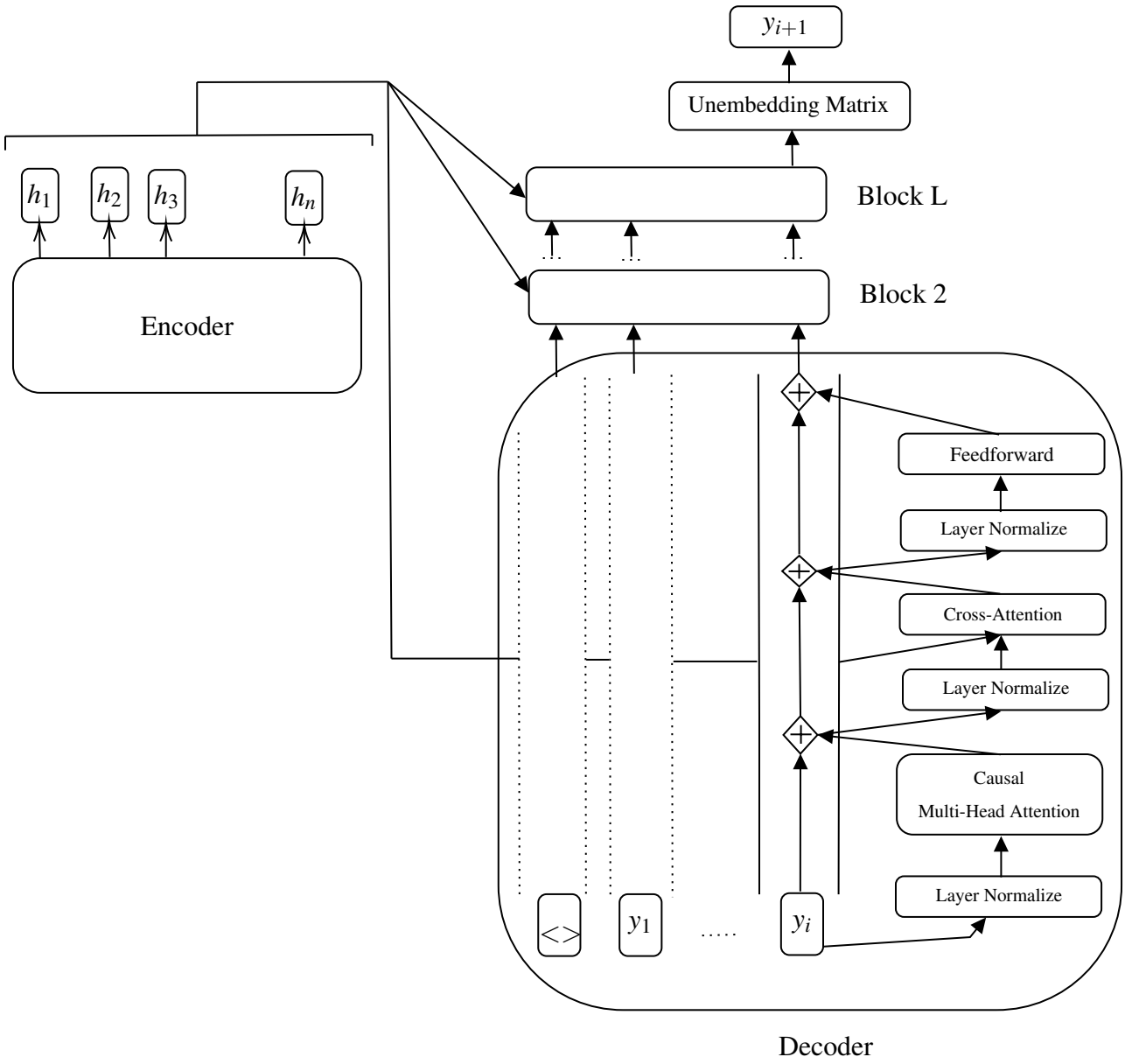


Figure 7: Overview of the Transformer Decoder architecture.

2.2.3 Convolutional Transformer

Unlike the previous model which employed a standard Vision Transformer as the decoder, the proposed architecture utilizes a Convolutional Transformer. This hybrid network integrates convolutional operators into the token embedding and attention mechanisms, effectively combining local feature extraction with the global attention capabilities of a standard Transformer architecture.

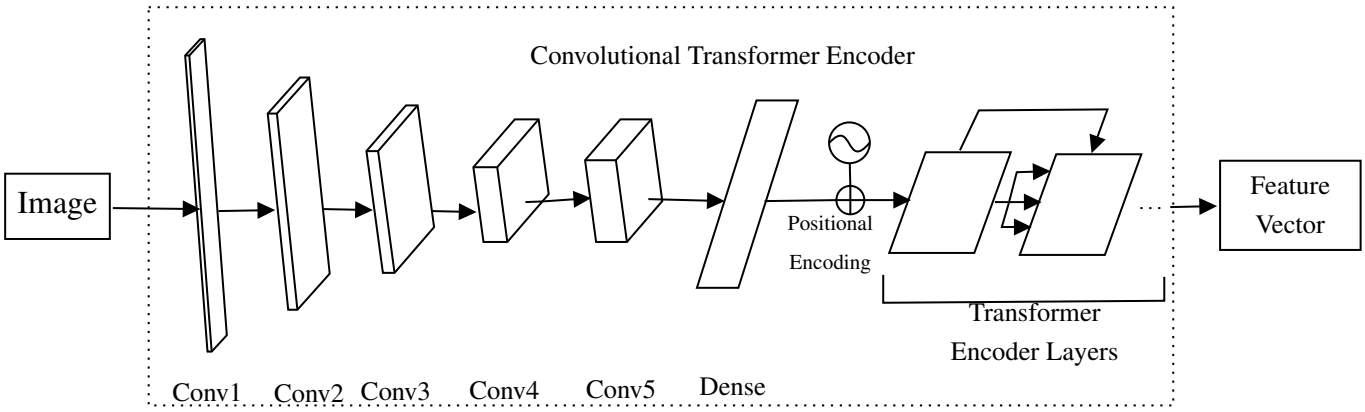


Figure 8: Overview of the Convolutional Transformer Encoder architecture.

2.3 Model Training

The model will be trained using a supervised learning approach, each training sample includes an image of a mathematical expression and its corresponding LaTeX sequence as the label.

2.3.1 Environment

The model was implemented using the PyTorch deep learning framework and trained on 50000 images using Google Colab Pro equipped with a single Nvidia A100 GPU.

2.3.2 Hyperparameter Settings

Hyperparameter configuration used for the reported results is summarized below:

Table 1: Hyperparameter settings used for model training.

Hyperparameter	**Value**	**Description**
Optimizer	Adam	Chosen for weight decay handling
Learning Rate	1×10^{-4}	Initial rate with scheduler
Batch Size	224	Adjusted for GPU memory
Max Epochs	100	Early stopping patience: 4
Input Resolution	50×200	Resized with padding
Max Sequence Length	152	Max LaTeX tokens generated
Dropout	0.1 - 0.3	Applied to prevent overfitting
Beam Width	5	Used during validation/inference

2.3.3 Loss Function

The model was trained to minimize the Cross-Entropy Loss between the predicted probability distribution and the ground truth LaTeX tokens. Padding tokens were ignored to prevent them from influencing the gradient updates.

2.3.4 Training Strategy

To ensure model convergence and prevent overfitting, the following strategies were adopted:

• **Teacher Forcing:** During training, instead of using predicted output from previous step, model receives ground truth output $\mathbf{y}(\mathbf{t})$ as input at time $\mathbf{t}+\mathbf{1}$, helping the model learn the correct structure faster. [1]

• **Early Stopping:** To prevent the model from overfitting to the training data, if the loss of validation set does not improve after 4 epochs, the training automatically stops. This ensures that the model weights are saved at the point of optimal generalization.

• **Dropout:** Dropout layers are inserted into encoder and decoder architectures (with probability rate from 0.1 to 0.3). By randomly "dropping out" neurons during training, he network is prevented from relying too heavily on specific features,thus improving the model's ability to generalize.

2.4 Model Testing

To generate the LaTeX sequence from the trained model, the beam search algorithm is used to generate a sequence of token ids before converting into a LaTeX sequence. Based on the original beam search algorithm [11], the beam search function can be constructed for the sequence generation problem, working step-by-step:

1. **Initiation:** The process starts with an initial token (initial beam).
2. **Candidate Generation:** In the first step, the model generates all possible next candidate tokens and calculates their probabilities.
3. **Top k Selection:** The algorithm selects the top k tokens with the highest probabilities as the candidate sequences (beams).
4. **Iteration & Expansion:** For each selected candidate beam, the model expands it by giving the probability of every single token in the vocabulary to be the next token. Then, the top k tokens with the highest probability of each beam are chosen as beam expansions, combining with the beam to create new longer sequences.
5. **Re-evaluation & Pruning:** The score for all newly generated sequences are calculated. After that, only the k candidate beams with the highest scores are chosen, being the candidate beams for the next step.
6. **Termination:** Step 4 & 5 are repeated until the end token in the vocabulary is generated or the number of iterations reaches the maximum sequence length.

In this project, the algorithm parameters are as below:

- **Beam width k:** A beam width of **5** is used, meaning there are 5 complete candidate sequences generated at the end of the process.
- **Start & end token:** The initial beam is the start token in the vocabulary or `<sos>`, while the end token is `<eos>`.
- **Maximum sequence length:** The maximum sequence length is **253**, which is also the maximum number of iterations.
- **Scoring function:** The initial beam has a score of 0. From step 1, after each candidate beam is expanded, the score is calculated by accumulating the score of the previous step candidate with the log-probability of the newly predicted token (beam expansion) and normalizing by the sequence length.

Compared to normal greedy decoding, in each step, instead of predicting only one token with the highest probability as the concrete guess, the beam search algorithm explores multiple possibilities as well as retains the best guesses from the previous step. This makes the process capable of recovering from the potential early mistakes and therefore produces better sequences with more accuracy.

Following the process above, there will be 5 candidate sequences in the final step. Finally, the sequence of tokens with the highest score is chosen before being decoded to be the final predicted LaTeX sequence.

2.5 Model Evaluation

This section describes the evaluation metrics used to assess model performance after testing. Two primary metrics are employed: Normalized Levenshtein and BLEU Score.

2.5.1 Token-level Levenshtein Distance

1. **Overview**

   The token-level Levenshtein distance measures the dissimilarity between a predicted token sequence P and a reference token sequence T by computing the minimum number of edit operations—token insertions, deletions, and substitutions—required to transform one sequence into the other [12, 13].

   Notation:

$$P = (p_1, p_2, \ldots, p_{|P|}) \quad \text{(predicted token sequence)}$$
$$T = (t_1, t_2, \ldots, t_{|T|}) \quad \text{(reference token sequence)}$$

   Each element p_i and t_j represents an individual token, such as a word, subword unit, or symbol.

2. **Computing the Token-level Levenshtein Distance**

   The computation follows a dynamic programming approach [12].

   Step 1: Matrix Initialization

   Let D be a matrix of size $(|P|+1) \times (|T|+1)$, where each entry $D_{i,j}$ represents the minimum number of token-level edit operations required to transform the first i tokens of P into the first j tokens of T.

   Initialize the base cases:

$$D_{0,j} = j \quad \text{for all } j, \qquad D_{i,0} = i \quad \text{for all } i$$

   Step 2: Dynamic Programming Calculation

   For each position $i \geq 1$ and $j \geq 1$, compute:

$$D_{i,j} = \min \begin{cases} D_{i-1,j} + 1 & \text{(token deletion)} \\ D_{i,j-1} + 1 & \text{(token insertion)} \\ D_{i-1,j-1} + c & \text{(token substitution)} \end{cases}$$

   where the substitution cost c is defined as:

$$c = \begin{cases} 0, & \text{if } p_i = t_j \quad \text{(tokens match)} \\ 1, & \text{if } p_i \neq t_j \quad \text{(tokens differ)} \end{cases}$$

Step 3: Final Distance

The token-level Levenshtein distance is obtained from the bottom-right cell of the matrix:

$$\text{LevDist}(P,T) = D_{|P|,|T|}$$

This value represents the minimum number of token edits required to transform P into T.

3. **Normalizing Score**

   To convert the distance into a normalized score ranging from 0 to 1, length-based normalization is applied [13]:

$$\text{Norm}(P,T) = \begin{cases} \dfrac{d}{L}, & \text{if } L > 0 \\ 1, & \text{if } L = 0 \end{cases}$$

   where:

$$\begin{aligned} d &= \text{LevDist}(P,T) \\ L &= \max(|P|,|T|) \quad \text{(number of tokens in the longer sequence)} \end{aligned}$$

4. **Averaging Across Multiple Sequences**

   For an evaluation set containing multiple prediction–reference pairs, average token-level metrics are computed [13]:

$$\text{AvgNorm} = \frac{1}{N}\sum_{i=1}^{N} \text{Norm}_i$$

   where N denotes the total number of sequence pairs in the evaluation dataset.

5. **Interpretation of the Normalized Score**

   The normalization process divides the edit distance d by the maximum length L of the predicted and reference sequences. As a result, the normalized score is constrained to the range $[0,1]$, which enables fair comparison between sequences of different lengths.

   A normalized value of 0 indicates a perfect match between P and T, meaning that no edit operations are required to transform one sequence into the other. Higher values represent greater dissimilarity between the two sequences.

   When $L = 0$, both sequences are empty. In this special case, the similarity is considered perfect, and the normalized score is set to 1 in order to avoid division by zero and maintain numerical stability.

2.5.2 BLEU Score

1. **Overview**

   BLEU (Bilingual Evaluation Understudy) is a widely used metric for evaluating the quality of predicted token sequences by measuring the overlap of token n-grams with one or more reference sequences [14]. By aggregating modified precision scores across multiple n-gram orders and applying a brevity penalty, BLEU captures both local token-level accuracy and global sequence-level structure. This makes it particularly suitable for sequence generation tasks where preserving multi-token patterns is essential.

   Notation:

$$P = (p_1, p_2, \ldots, p_{|P|}) \quad \text{(predicted token sequence)}$$
$$T = (t_1, t_2, \ldots, t_{|T|}) \quad \text{(reference token sequence)}$$

   An **n-gram** is defined as a contiguous sequence of n tokens extracted from a tokenized sequence.

2. **Modified N-gram Precision**

   The BLEU score is constructed using modified n-gram precision, which measures how many predicted n-grams appear in the reference sequence while preventing overestimation due to repeated n-grams [14].

   Step 1: N-gram Extraction

   For a given n-gram order k, all contiguous k-grams are extracted from both sequences:

$$\text{ngrams}_k(P) = \{(p_i, \ldots, p_{i+k-1}) \mid 1 \leq i \leq |P| - k + 1\}$$
$$\text{ngrams}_k(T) = \{(t_i, \ldots, t_{i+k-1}) \mid 1 \leq i \leq |T| - k + 1\}$$

   Step 2: Precision Calculation

   Let $C_P^{(k)}(x)$ and $C_T^{(k)}(x)$ denote the frequency of an n-gram x in the predicted and reference sequences, respectively. The modified precision for k-grams is computed as [14]:

$$P_k = \frac{\sum_x \min\left(C_P^{(k)}(x), C_T^{(k)}(x)\right)}{\sum_x C_P^{(k)}(x)}$$

   This formulation clips the predicted n-gram counts by their maximum occurrences in the reference sequence, ensuring that repeated predictions do not receive disproportionate credit.

3. **Geometric Mean of Precisions**

   As the n-gram order increases, precision values generally decrease due to stricter matching requirements. BLEU mitigates this effect by combining modified precisions across multiple n-gram orders using a geometric mean [14]:

$$\text{GeoMean} = \exp\left(\frac{1}{N}\sum_{k=1}^{N} \ln P_k\right)$$

where N denotes the maximum n-gram order considered, typically set to $N = 4$.

This aggregation strategy:

- Assigns equal importance to each n-gram order
- Balances short-range and long-range token dependencies
- Prevents lower-order n-grams from dominating the final score

Note: If any modified precision $P_k = 0$, the geometric mean becomes zero.

4. **Brevity Penalty**

   Since n-gram precision alone can favor shorter predictions, BLEU incorporates a brevity penalty to discourage overly short sequences [14]:

$$BP = \begin{cases} 1, & \text{if } |P| \geq |T| \\ \exp\left(1 - \dfrac{|T|}{|P|}\right), & \text{if } |P| < |T| \end{cases}$$

   The brevity penalty reduces the BLEU score when the predicted sequence is shorter than the reference, encouraging outputs that more closely match the reference length.

5. **Final BLEU-N Score**

   The final BLEU score combines the geometric mean of precisions with the brevity penalty [14]:

$$\text{BLEU}_N = BP \cdot \text{GeoMean}$$

6. **Interpretation of the BLEU Score**

   The BLEU score combines the geometric mean of n-gram precision values with a brevity penalty to evaluate both translation accuracy and length adequacy.

   The resulting BLEU score ranges from 0 to 1, where higher values indicate better agreement between the predicted output and the reference text in terms of both content and structure.

2.6 Website Implementation

In this section, we aim to integrate the trained models into a web application. The proposed system is developed as a web-based application using a Micro-service architecture. It consists of four core components including the development framework and technology stack, backend implementation, frontend interaction and asynchronous handling, and system workflow.

2.6.1 Development Framework and Technology Stack

The system is built upon the Flask micro-framework (Python 3.x), chosen for its lightweight footprint and seamless integration with PyTorch. On the client side, standard web technologies (HTML5, CSS3, and Vanilla JavaScript) are used to ensure cross-platform compatibility. Additionally, the MathJax 3 library is integrated via CDN to dynamically render the raw LaTeX strings returned by the models into high-quality vector mathematical equations.

2.6.2 Backend Implementation (Server-Side)

The core logic is encapsulated in the `app.py` module. To optimize performance and address the domain-specific challenges of handwritten mathematical expression recognition, the backend implements the following strategies:

1. **Global Resource Initialization**: The pre-trained model weights and vocabulary file (`vocab.json`) are loaded into memory strictly once upon server startup. This "Load-Once" strategy eliminates the overhead of reloading neural networks for every request, significantly reducing inference latency.

2. **Preprocessing Pipeline**: To handle variable input resolutions without distorting the mathematical characters, raw images undergo a rigorous transformation sequence: (1) Grayscale conversion; (2) Resizing to target dimensions of 200×50 pixels (width $\times$ height) using bilinear interpolation with anti-aliasing; (3) Alpha channel handling with white background compositing; and (4) Tensor normalization to $[0, 1]$ range.

3. **Inference Engine**: The system utilizes a Greedy Search decoding algorithm. The decoder generates tokens sequentially by selecting the highest probability output at each time step until the `<eos>` token is produced.

2.6.3 Frontend Interaction and Asynchronous Handling

The user interface ensures a responsive experience by utilizing the JavaScript Fetch API. Image data is transmitted via HTTP POST requests asynchronously (AJAX), preventing the web page from freezing during server computations. The interface supports drag-and-drop file uploads and validates image formats (PNG, JPG, JPEG, GIF, BMP, WebP) with a maximum size of 5MB. Upon receiving the JSON response, the frontend updates the DOM and triggers MathJax to visualize the equation.

2.6.4 System Workflow

The end-to-end execution flow follows this sequence:

1. **Input**: User uploads an image via the web interface.

2. **Transmission**: Frontend encapsulates data in a `FormData` object and sends it to the `/predict` endpoint.

3. **Processing**: Server preprocesses the image and runs the Encoder-Decoder model.

4. **Decoding**: The model's output indices are mapped to LaTeX strings via the vocabulary.

5. **Rendering**: The browser displays the final mathematical equation.

3 Results

3.1 Model Parameter Comparison

Model	Number of Learnable Parameters
CNN–LSTM	4,420,297
Vision Transformer (ViT)	27,836,105
Convolutional Transformer	19,091,321

Table 2: Number of learnable parameters for each model

The number of parameters of each model is illustrated in Table 2. The CNN–LSTM model is the most lightweight architecture, with 4.4 millions parameters, which can be attributed to the use of convolutional layers for spatial feature extraction and recurrent layers for sequence modeling. In contrast, the transformer-based models are substantially larger. The Vision Transformer contains 27.8 millions parameters, as it relies entirely on self-attention mechanisms and large projection matrices to model global dependencies. The Convolutional Transformer reduces the parameter count to 19.1 millions by integrating convolutional operations into the transformer architecture. This design introduces local feature modeling and hierarchical representations, which decreases the need for extensive self-attention and results in a more parameter-efficient alternative to a pure Vision Transformer.

3.2 Model Training and Validation Results

1. **CNN-LSTM Model**

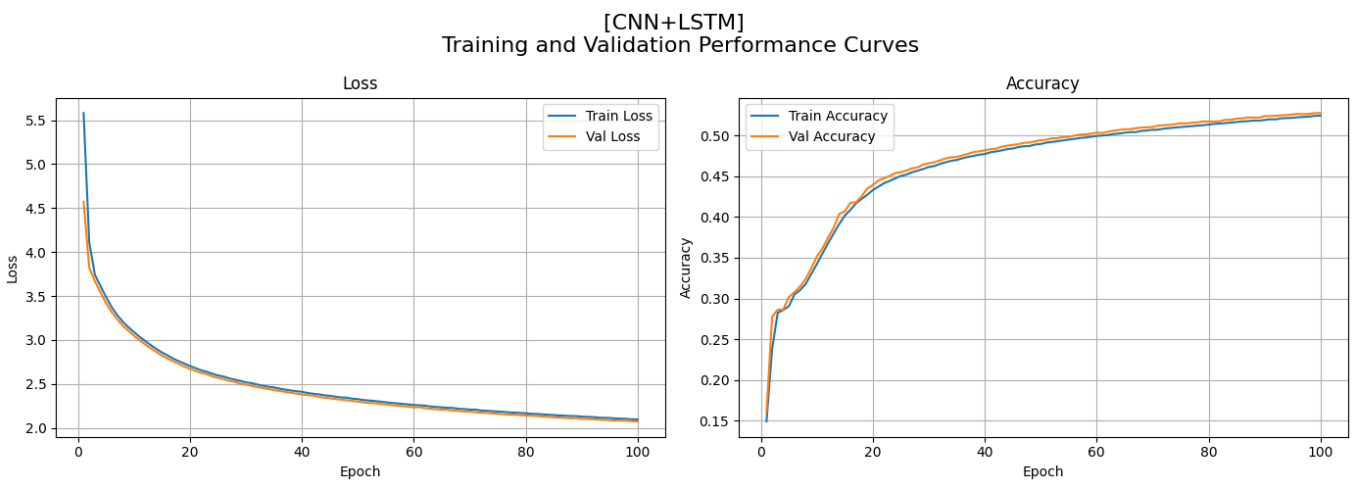


Figure 9: CNN-LSTM Model: Training and Validation Performance Curves

- Figure 9 presents the training and validation loss and accuracy curves for the CNN-LSTM model over 100 epochs. The training loss decreases rapidly in the first few epochs and then gradually converges, while the validation loss follows the same trend throughout training. This demonstrates stable optimization behavior.
- In terms of accuracy, both training and validation accuracy steadily increase and remain close to each other, reaching approximately 53% at the final epoch. The small gap between the two curves suggests that the model generalizes consistently to unseen data.

2. **Vision Transformer (ViT) Model**

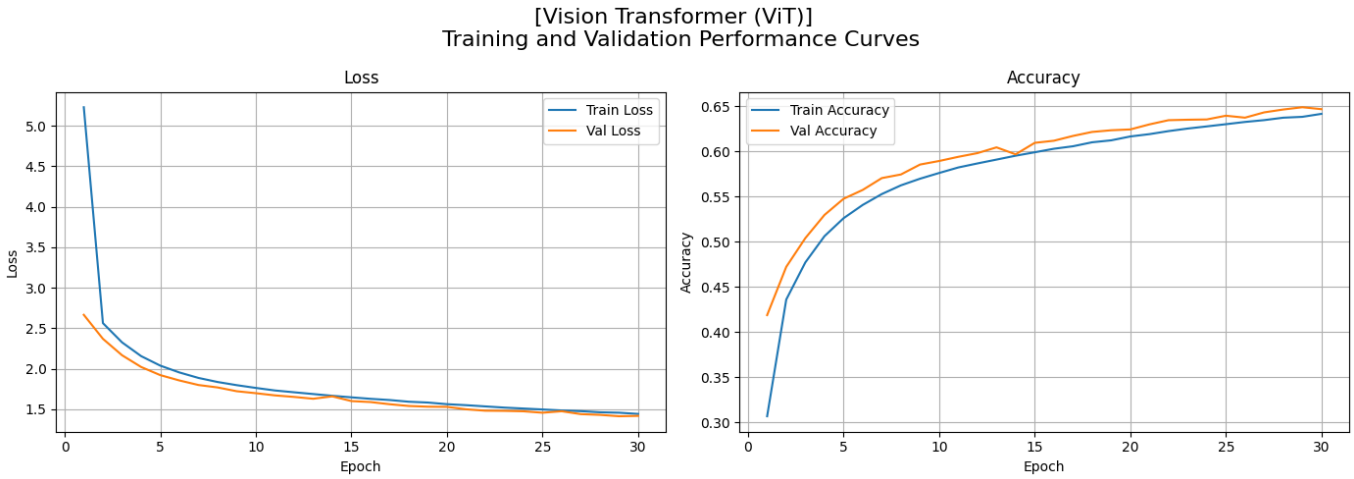


Figure 10: ViT Model: Training and Validation Performance Curves

- As shown in Figure 10, the ViT converges significantly faster than the CNN-LSTM model, reaching stable performance in approximately 30 epochs. Both training and validation loss decrease smoothly, with validation loss remaining slightly lower than training loss throughout training, suggesting good regularization and generalization.
- The accuracy curves show a clear improvement over CNN+LSTM, with validation accuracy reaching approximately 65%. The small difference between training and validation accuracy indicates minimal overfitting.

3. **Convolutional Transformer Model**

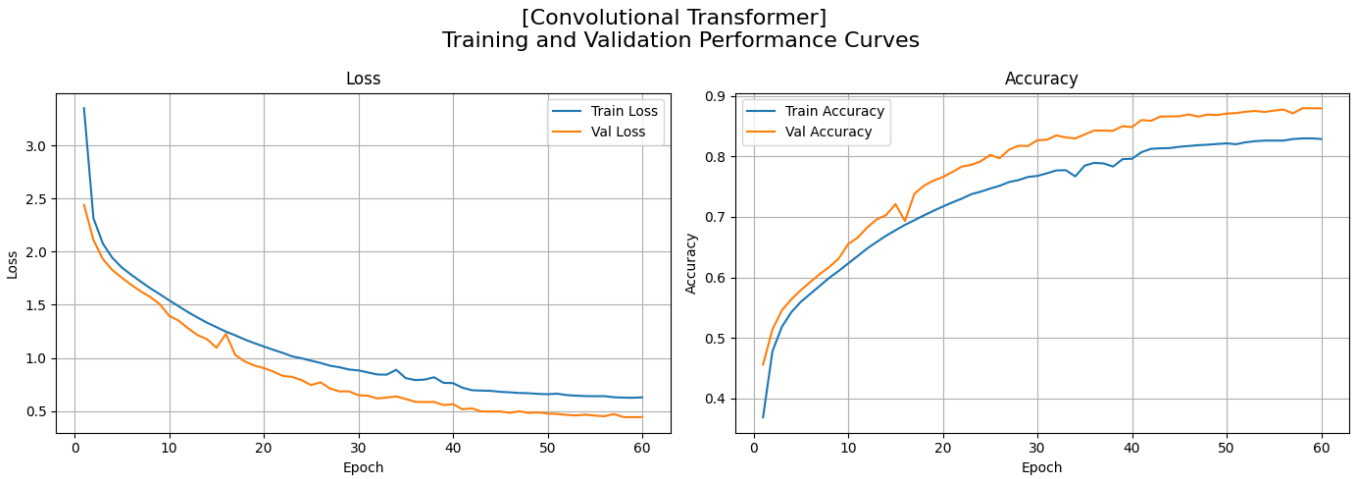


Figure 11: Convolutional Transformer Model: Training and Validation Performance Curves

- Figure 11 illustrates the training behavior of the Convolutional Transformer. The model shows a consistent decrease in both training and validation loss across 60 epochs, with validation loss decreasing more rapidly at a lower value than training loss. This behavior suggests strong generalization and robust feature learning.
- Convolutional Transformer achieves the highest performance among all models, with validation accuracy steadily increasing and reaching approximately 88%. The validation accuracy remains

stable and consistently higher than the training accuracy, indicating effective regularization and reduced overfitting. Overall, Convolutional Transformer demonstrates superior convergence behavior and classification capability.

4. **Comparison**

   Overall, the Convolutional Transformer outperforms both CNN-LSTM and ViT models in terms of accuracy and generalization. While CNN-LSTM shows stable training behavior, its performance is limited by lower accuracy. ViT improves substantially upon CNN-LSTM, achieving better accuracy with faster convergence. However, Convolutional Transformer achieves the best balance between convergence speed, stability, and final performance, making it the most effective architecture for this task.

3.3 Test Set Evaluation

After testing three models with the test set containing 10355 images, we applied two metrics: Levenshtein Distance and BLEU Score, to evaluate models. These metrics quantify how closely the predicted sequences match the reference sequences:

- Normalized Levenshtein Distance measures the cost of editing the predicted LaTeX sequences to get the reference sequences, normalized by sequence length. The value range for this metric is [0,1]. Lower values indicate better performance.
- BLEU Score evaluates n-gram overlap between the predicted and reference sequences. The value range for this metric is [0,1]. Higher values indicate better predictions.

Table 3: Test Set Evaluation

Model	**Levenshtein Distance**	**BLEU Score**
CNN-LSTM	0.8765	0.0379
Vision Transformer (ViT)	0.7347	0.0958
Convolutional Transformer	0.3233	0.5405

The result is illustrated in the Table 3. From the result, it can be observed that:

- The CNN-LSTM model performed the worst among the three models, showing the highest normalized Levenshtein Distance and the lowest BLEU score. This indicates that the older architecture like CNN and LSTM struggled to predict the LaTeX sequences compared to more recent transformer-based architectures.
- The ViT model achieved moderate performance, improving over CNN-LSTM but still far from the best-performing model.
- The Convolutional Transformer model outperformed both CNN-LSTM and ViT by a large distance, achieving the lowest normalized Levenshtein distance and the highest BLEU score, indicating its superior ability to generate LaTeX sequences closely matching the ground truth.

3.4 Model Deployment Results on Website

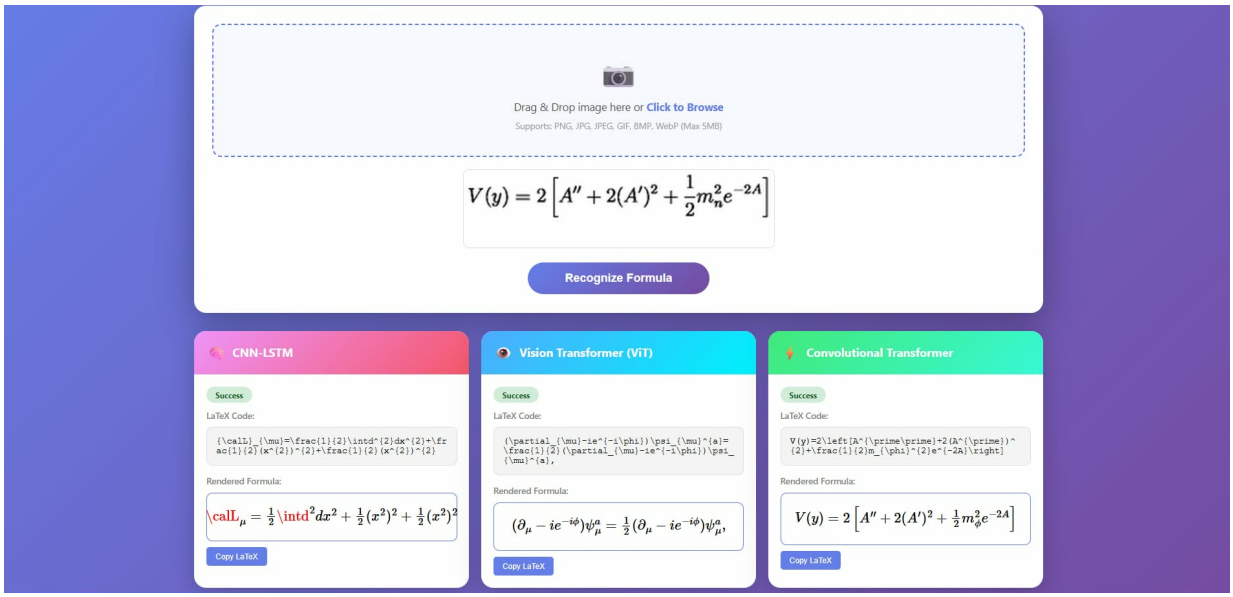


Figure 12: Web application interface displaying prediction results from three models: CNN-LSTM, ViT, and Convolutional Transformer.

3.4.1 Website Overview

We successfully deployed the trained models on a web-based platform that allows users to interact directly with the models. The website provides an UI where users can upload an image containing a mathematical formula by drag-and-drop or file browsing. Once uploaded, the system processes the image and returns the LaTeX code of the formula.

The deployed system integrates our three models. Each model works independently, and their outputs are displayed together to allow direct comparison of recognition results.

3.4.2 User Interaction

The interaction pipeline is as follows:

1. Upload a formula image through the web interface.
2. Trigger the recognition process by "Recognize Formula" button.
3. Receive the predicted LaTeX code from each model.
4. Visualize the rendered mathematical expression corresponding to the predicted LaTeX.
5. Optionally copy the generated LaTeX code for external use.

This design enables real-time inference and immediate feedback, demonstrating the practicality of the deployed models beyond offline evaluation.

3.4.3 Example Result

To validate the deployed system, we conducted inference testing using a sample mathematical formula. The ground truth expression is:

$$V(y) = 2\left[A'' + 2(A')^2 + \frac{1}{2}m_n^2 e^{-2A}\right]$$

Table 4: Prediction Results Comparison

Model	**Status**	**Observation**
CNN-LSTM	Incorrect	Generated unrelated Lagrangian expression
Vision Transformer (ViT)	Incorrect	Produced erroneous partial differential equation
Convolutional Transformer	Mostly Correct	Minor error: predicted m_ϕ instead of m_n

As illustrated in Figure 12, the results demonstrate a clear performance hierarchy among the three architectures. The Convolutional Transformer model successfully recognized the formula structure, bracket hierarchy, and most symbols, achieving approximately 96% character-level accuracy. In contrast, both CNN-LSTM and ViT models exhibited complete recognition failure, generating semantically unrelated mathematical expressions.

This outcome aligns with our theoretical expectations: the hybrid CNN-Transformer architecture of Convolutional Transformer effectively leverages local feature extraction while maintaining global contextual understanding, proving most suitable for mathematical formula recognition tasks under limited training data conditions.

4 Discussion

4.1 On the Success Rate of the Model

The evaluation results reveal clear differences in performance among the three models. The CNN-LSTM model consistently underperforms both Transformer-based architectures, exhibiting the worst performance in both training and validation results and applying metrics on test set. We suspect the problem associated with the model lies in the training process: Since only one cell vector is in charge of keeping all information of the image (in contrast to Vision Transformer and Convolutional Transformer), information loss happens as the length of the sequence increases. The idea of information loss is mentioned in [1]. To resolve the problem, deploying an Attention model might work better. Furthermore, the number of parameters is significantly lower in this model in comparison to the two other ones. This leads to a hypothesis: Will increasing the number of parameters significantly improve the LaTeX generation accuracy?

The ViT model demonstrates improved performance over CNN-LSTM. Faster convergence and higher validation accuracy suggest that the attention mechanism can capture global patterns in the input data. However, its performance remains bad, indicated by the compiled code and two metrics used: BLEU and Levenstein. We suspect this is due to the low number of epochs in the model. In fact, Figure 10 shows that the model is already converging, making continuous training no longer worthwhile.

The Convolutional Transformer model clearly outperforms both CNN-LSTM and ViT. By combining CNN layers with Vision Transformer, the model captures both local and global features effectively. This results in the lowest normalized Levenshtein distance, highest BLEU score and best accuracy on training/validation set, highlighting the model's superior power of sequence prediction and generalization. Overall, the results indicate that hybrid architectures like Convolutional Transformers achieve the highest success rate, while traditional CNN-LSTM models are less effective for this task.

4.2 Future Perspectives

These findings suggest several directions for future work. First, hybrid architectures that combine local and global feature modeling, similar to Convolutional Transformer, are promising and could be further optimized for even better performance. Second, improving training strategies, such as data augmentation, learning rate scheduling, or larger pre-training datasets, may enhance the performance of both traditional and transformer-based models. Finally, exploring alternative architectures that integrate convolutional, recurrent, and attention mechanisms could lead to models that are both efficient and highly accurate in sequence prediction tasks. Such advancements could further reduce errors and improve generalization across more complex datasets.

5 Summary

In this report, we explored and compared the performance of three architectures: CNN-LSTM, Vision Transformer (ViT), and Convolutional Transformer, for LaTeX generation from images tasks. Each model was evaluated in terms of training and validation behavior, as well as test set performance using Levenshtein distance and BLEU score. The results demonstrate clear differences among the models.

Overall, the study highlights the importance of model architecture in LaTeX generation from images tasks, and OCR problems in general. Transformer-based models, particularly hybrid architectures like Convolutional Transformer, offer significant advantages in terms of accuracy, generalization, and robustness. Future work may explore additional hybrid architectures, optimized training strategies, or larger datasets to further enhance predictive performance and reduce errors.

References

[1] Daniel Jurafsky and James H. Martin. 2025. "Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models, 3rd edition." Online manuscript released August 24, 2025. https://web.stanford.edu/~jurafsky/slp3

[2] A. Vaswani, et al. "Attention is all you need". Available: https://arxiv.org/pdf/1706.03762

[3] Alexey Dosovitskiy et al. "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale". Available: https://arxiv.org/pdf/2010.11929

[4] Mozhdeh Gheini, Xiang Ren and Jonathan May, "Cross-Attention is All You Need: Adapting Pretrained Transformers for Machine Translation". Available: https://arxiv.org/abs/2104.08771

[5] Jayaprakash Sundararaj et al. "Automated LaTeX Code Generation from Handwritten Mathematical Expressions". Available: https://arxiv.org/pdf/2412.03853

[6] Killian Barrere et al. "A Light Transformer-Based Architecture for Handwritten Text Recognition". Available: https://hal.science/hal-03685976/file/A_Light_Transformer_Based_Architecture_for_Handwritten_Text_Recognition.pdf

[7] Im2LaTeX dataset https://www.kaggle.com/datasets/shahrukhkhan/im2latex100k?select=im2latex_formulas.norm.csv

[8] "re" library https://docs.python.org/3/library/re.html

[9] "torchtext.Vocab" library https://docs.pytorch.org/text/master/vocab.html

[10] "torch.nn" library version 2.3.0. Document available: https://docs.pytorch.org/docs/2.3/nn.html#module-torch.nn

[11] M. Freitag and Y. Al-Onaizan, “Beam Search Strategies for Neural Machine Translation,” in Proceedings of the First Workshop on Neural Machine Translation, Vancouver, Canada, Aug. 2017, pp. 56–60, Association for Computational Linguistics. Available: https://aclanthology.org/W17-3207.pdf

[12] Rishin Haldar and Debajyoti Mukhopadhyay. "Levenshtein Distance Technique in Dictionary Lookup Methods: An Improved Approach". Available: https://arxiv.org/pdf/1101.1232

[13] Carmine Gravino, Alessandra Orsi, Michele Risi. "Using the Normalized Levenshtein Distance to Analyze Relationship between Faults and Local Variables with Confusing Names: A further Investigation". Available: https://ksiresearch.org/seke/seke21paper/paper124.pdf

[14] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. "BLEU: a Method for Automatic Evaluation of Machine Translation". Available: https://aclanthology.org/P02-1040.pdf

Appendix: Source Code Repository

The complete source code used in this project is available at the following GitHub repository: `https://github.com/watermelon-3012/gp_image_to_latex`

The repository includes the following files:

- `README.md` – Introduction of the project
- `all_in_one_experiment.ipynb` – Contains everything from data preprocessing, model implementation and performance evaluation, and metrics.
- `Web Implementaion` – The folder containing source code for the web implementation.
- `G32-GP2026-ICT.pdf` – The report for this project.